

Data structures

QUICK SORT

LOWER BOUNDS

Quicksort High-Level Idea

Divide: Partition the array into left and right sub-arrays.

- Choose an element of the array, called a **pivot**.
- Put elements in the left sub-array if $\leq$ pivot.
- Put elements in the right sub-array if $>$ pivot.

Conquer: Recursively sort the sub-arrays on the left and the right of the pivot.

The base case: a **single element** is already sorted (or sort a small constant number of elements).

Recursive Quicksort Pseudocode

```
Quicksort(A, l, r)
  if r - l > 2
    p ← Partition(A, l, r)
    Quicksort(A, l, p-1)
    Quicksort(A, p+1, r)
  else
    SimpleSort(A, l, r)
```

A = array, l = left index, r = right index, p = pivot index

To sort the entire array A , the initial call is **QUICKSORT**(A , 1, $A.length$)

Another Algorithm for Partition

Partition(A, l, r) is a procedure that rearranges the subarray $A[l \dots r]$ (Starting in $A[l]$, ending in $A[r]$) in-place.

PARTITION(A, l, r)

```
1   $x = A[r]$ 
2   $i = l - 1$ 
3  for  $j = l$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6      exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 
```

- First, we grab the last item in the list. This item is the pivot.
- We'll move the pointer from left to the right until we get to an item that's larger than the pivot.
- Time to swap (and repeat).
- Perform the last swap.
- Return an index value that represents the position of the pivot element after the partition step is completed.

Partition Example

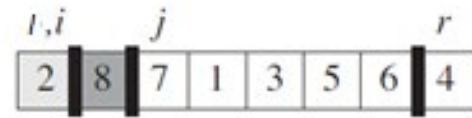
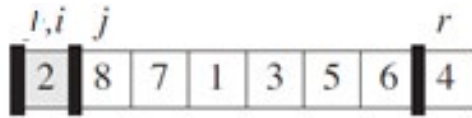
l, i	j						r
2	8	7	1	3	5	6	4

	i	j
Less than pivot	Greater than pivot	Waiting for partition

PARTITION(A, l, r)

```
1   $x = A[r]$ 
2   $i = l - 1$ 
3  for  $j = l$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 
```

Partition Example

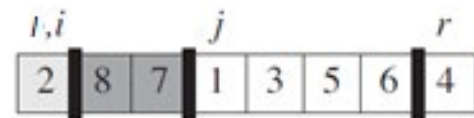
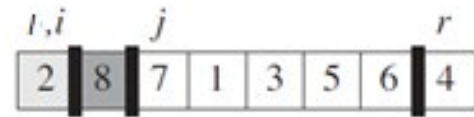
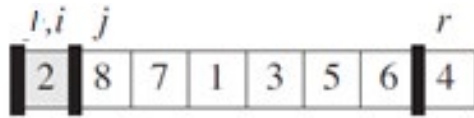


	i	j
Less than pivot	Greater than pivot	Waiting for partition

PARTITION(A, l, r)

- 1 $x = A[r]$
- 2 $i = l - 1$
- 3 for $j = l$ to $r - 1$
- 4 if $A[j] \leq x$
- 5 $i = i + 1$
- 6 exchange $A[i]$ with $A[j]$
- 7 exchange $A[i + 1]$ with $A[r]$
- 8 return $i + 1$

Partition Example



i

j

Less than
pivot

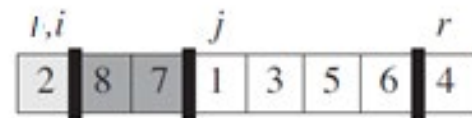
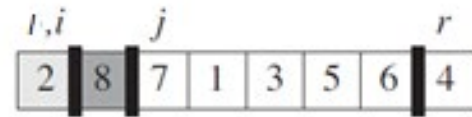
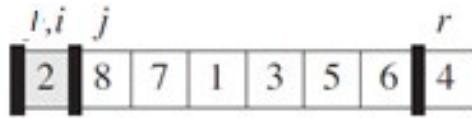
Greater than
pivot

Waiting for
partition

PARTITION(A, l, r)

- 1 $x = A[r]$
- 2 $i = l - 1$
- 3 for $j = l$ to $r - 1$
- 4 if $A[j] \leq x$
- 5 $i = i + 1$
- 6 exchange $A[i]$ with $A[j]$
- 7 exchange $A[i + 1]$ with $A[r]$
- 8 return $i + 1$

Partition Example

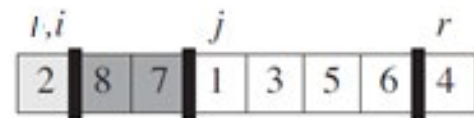
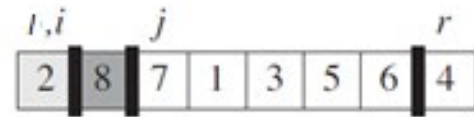
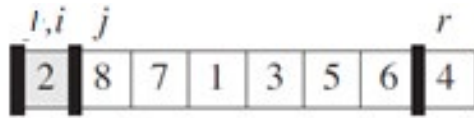


	i	j
Less than pivot	Greater than pivot	Waiting for partition

PARTITION(A, l, r)

- 1 $x = A[r]$
- 2 $i = l - 1$
- 3 for $j = l$ to $r - 1$
- 4 if $A[j] \leq x$
- 5 $i = i + 1$
- 6 exchange $A[i]$ with $A[j]$
- 7 exchange $A[i + 1]$ with $A[r]$
- 8 return $i + 1$

Partition Example

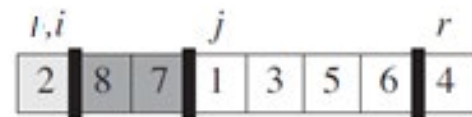
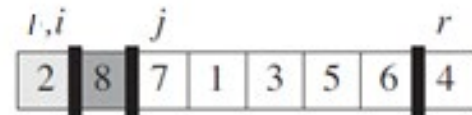
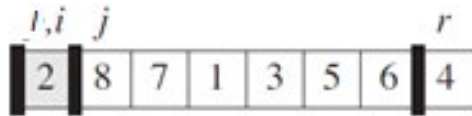


	i	j
Less than pivot	Greater than pivot	Waiting for partition

PARTITION(A, l, r)

- 1 $x = A[r]$
- 2 $i = l - 1$
- 3 for $j = l$ to $r - 1$
- 4 if $A[j] \leq x$
- 5 $i = i + 1$
- 6 exchange $A[i]$ with $A[j]$
- 7 exchange $A[i + 1]$ with $A[r]$
- 8 return $i + 1$

Partition Example

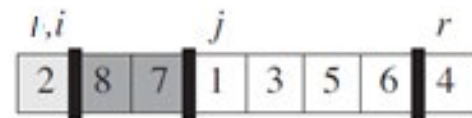
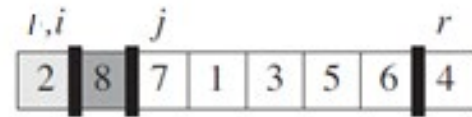
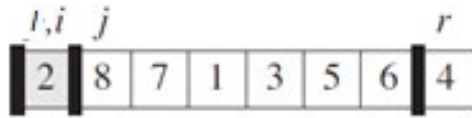


	i	j
Less than pivot	Greater than pivot	Waiting for partition

PARTITION(A, l, r)

- 1 $x = A[r]$
- 2 $i = l - 1$
- 3 for $j = l$ to $r - 1$
- 4 if $A[j] \leq x$
- 5 $i = i + 1$
- 6 exchange $A[i]$ with $A[j]$
- 7 exchange $A[i + 1]$ with $A[r]$
- 8 return $i + 1$

Partition Example

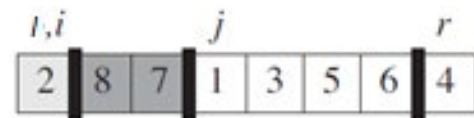
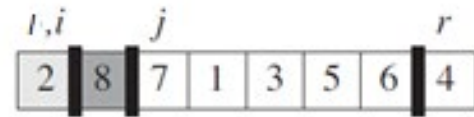
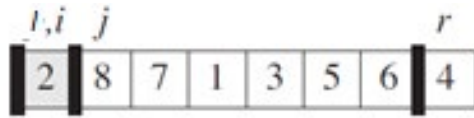


	i	j
Less than pivot	Greater than pivot	Waiting for partition

PARTITION(A, l, r)

- 1 $x = A[r]$
- 2 $i = l - 1$
- 3 for $j = l$ to $r - 1$
- 4 if $A[j] \leq x$
- 5 $i = i + 1$
- 6 exchange $A[i]$ with $A[j]$
- 7 exchange $A[i + 1]$ with $A[r]$
- 8 return $i + 1$

Partition Example



	i	j
Less than pivot	Greater than pivot	Waiting for partition

PARTITION(A, l, r)

- 1 $x = A[r]$
- 2 $i = l - 1$
- 3 for $j = l$ to $r - 1$
- 4 if $A[j] \leq x$
- 5 $i = i + 1$
- 6 exchange $A[i]$ with $A[j]$
- 7 exchange $A[i + 1]$ with $A[r]$
- 8 return $i + 1$

Quicksort Complexity

Run Time complexity:

- **Average case:** $O(n \log n)$

Space complexity:

- $O(\log n)$ additional space on average – for the recursion stack.
 - Better than merge sort's $O(n)$ additional space complexity.

Quicksort Worst Case Analysis

1. Array is already sorted in increasing order.
 2. Array is already sorted in decreasing order.
- **NOTE:** all of the above hold as worst case for choosing the pivot as the first/last element, other versions of Quicksort have better methods for picking the pivot, but can still have $\theta(n^2)$ run time for certain inputs.

More formally: $T(n) = T(n - 1) + \theta(n) = O(n^2)$.

Quicksort Complexity

Run Time complexity:

- **Average case:** $O(n \log n)$ – i.e., the best run time possible for a comparison-based sorting algorithm.
- **Worst case:** $O(n^2)$ ☹️.
- **Best case:** $O(n \log(n))$ – More on this later.
- In practice, quicksort is usually the fastest comparison-based algorithm.

Space complexity:

- $O(\log n)$ additional space on average – for the recursion stack. Better than merge sort's $O(n)$ space complexity.

Question 1

What is the run time of quicksort when all elements are the same?

Question 1 - Solution

What is the run time of quicksort when all elements are the same?

Solution:

We can think about such an array as already being sorted. For our way of choosing the pivot, this is the worst case. So, the runtime is $\theta(n^2)$.

Question 2

Here is an array that has just been partitioned for the first step of quicksort:

[3, 0, 2, 4, 6, 8, 7, 9, 7]

Which of the numbers in the array could be the pivot of this partition?

Question 2 - Solution

Here is an array that has just been partitioned for the first step of quicksort:

[3, 0, 2, 4, 6, 8, 7, 9, 7]

Which of the numbers in the array could be the pivot of this partition?

Solution

4, 6

Any number at index p where:

- for all $i < p$, $A[i] \leq A[p]$
- For all $j > p$, $A[j] > A[p]$

Quicksort Summary

Strengths:

- **Fast.** On average, quicksort runs in $O(n\log(n))$ time, which scales well as n grows.
- **Parallelizable.** Quicksort divides the input into two independent sections, that can be sorted simultaneously.

Weaknesses:

- **Slow Worst-Case.** In the worst case, quicksort can take $O(n^2)$ time. While this isn't common, it makes quicksort undesirable in cases where any slow performance is unacceptable.

[One such case is the Linux kernel. They [use heapsort instead of quicksort](#), so viruses and hackers can't exploit the slow worst-case time of sorting to tie up a machine's processor (and because heapsort has a lower space cost).]

Lower bounds

Upper bound:

Proved by showing the **existence of an algorithm** for the problem and analyzing its complexity.

Lower bound:

Proved by presenting a proof that applies to **every algorithm** for the problem.

Lower bounds

In other words:

If the **lower bound** for the worst (respectively, average / best) case problem is $L(n)$, then every algorithm that solves this problem must take $\Omega(L(n))$ time in the worst (respectively, average / best) case.

Claim: Any algorithm that solves the sorting problem must read the entire input.

Proof:

- Suppose there exists a sorting algorithm that doesn't read all input elements.
- There must be some position i in the input array that is never read by the algorithm.
- So, the output of the algorithm on two inputs that differ only at position i must be the same.
- Consider two inputs that differ only at position i :
 - Input A : element i is greater than all other elements
 - Input B : element i is smaller than all other elements
- But the correct sorted order would be different for A and B .
- This contradicts the assumption that the algorithm correctly sorts all inputs.

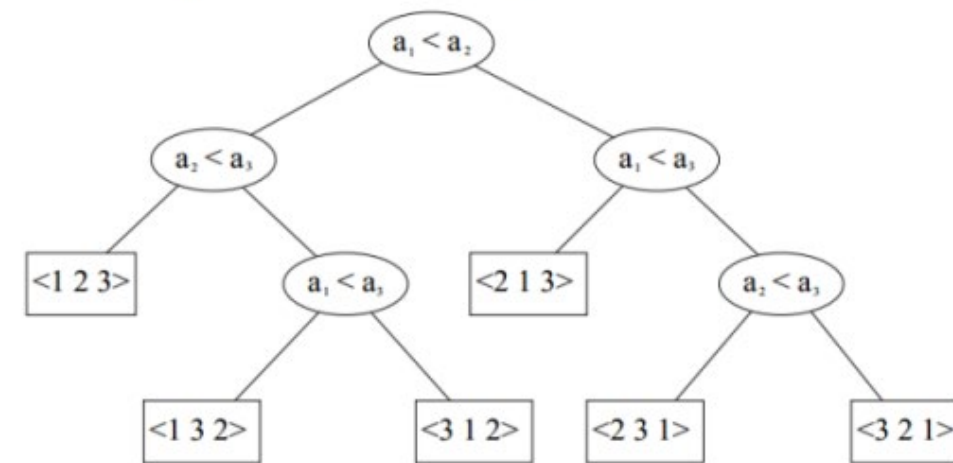
Therefore, any correct sorting algorithm must examine every element of the input array, giving us a lower bound of $\Omega(n)$ for any sorting algorithm.

Comparison Based Sorting – $\Omega(n \cdot \log n)$ Lower bound (worst case)

- We have seen in class that for every comparison-based sorting algorithm, there exists an input on which the algorithm performs $\Omega(n \cdot \log n)$ comparisons.
- Idea of the proof (reminder):
 - Model the algorithm as a binary decision tree such that each node has 0 or 2 children.
 - Count the number of leaves in the tree. $n!$
 - Relate the number of leaves to the height of the tree.

$$\log(n!) = \Theta(n \log(n))$$

Binary Decision Tree for 3 Elements



Question 3

Alice picks a number between 1 and n , and Bob must guess it by asking yes/no questions (e.g., “Is it greater than 10?”). Alice answers truthfully, and Bob must eventually guess the exact number.

We need to show that for any strategy used by Bob, Alice can choose a number that forces Bob to ask $\Omega(\log n)$ questions.

Question 3 - Solution

Bob's strategy can be thought of as a **binary decision tree**:

- Each **node** is a question Bob asks (e.g., "Is the number greater than x ?").
- The two **children** of a node represent Alice's answers: "yes" or "no."
- A **leaf** is where Bob makes a guess (e.g., "The number is 5").
- Since Bob must guess any number from 1 to n , there must be at least n leaves - one for each possible number.

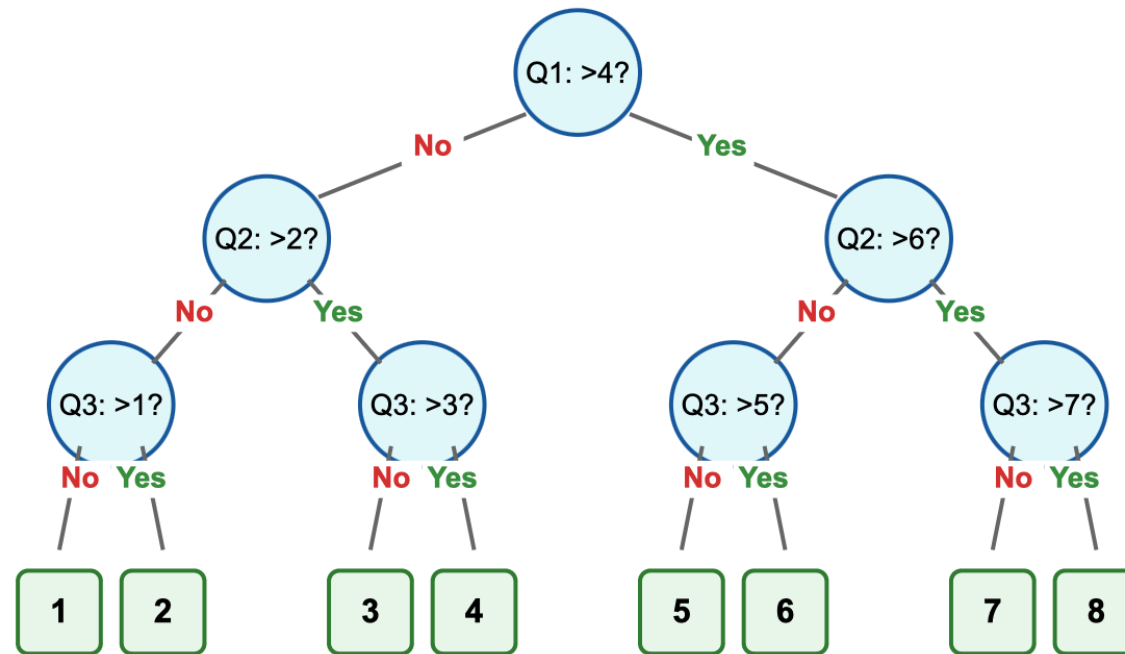
Question 3 - Solution

The Tree's Structure

- Start with all numbers $\{1, 2, \dots, n\}$.
- Bob asks a question, splitting the numbers into two groups (e.g., “Is it greater than $\frac{n}{2}$?” splits into $\{1, \dots, \frac{n}{2}\}$ and $\{\frac{n}{2} + 1, \dots, n\}$).
- For each answer, Bob asks another question, further splitting the groups, until he narrows it down to one number (a leaf).

Question 3 - Solution

Example for $n = 8$, Alice picks a number between 1-8, Bob must find it.



Key Insights:

- Tree height = $\log_2(n) = \log_2(8) = 3$ questions needed in worst case
- Alice can force Bob to use all 3 questions by choosing strategically

Question 3 - Solution

Relate the Number of Leaves to the Height

- A binary tree with height h can have at most 2^h leaves (if perfectly balanced).
- We need at least n leaves to cover all numbers from 1 to n : $2^h \geq n \implies h \geq \log n$
- Since h must be an integer, the minimum height is $\lceil \log n \rceil$.

Interpret the Height

- The height h of the tree is the number of questions Bob asks on the longest path from the root to a leaf.
- Alice can always pick a number corresponding to the deepest leaf, forcing Bob to ask the maximum number of questions.
- For example, in the $n = 8$ case, if Alice picks 5, Bob asks 3 questions, which is $\log 8$.

Question 3 - Solution

Worst-Case Lower Bound

- For any strategy Bob uses (any binary decision tree), the tree must have at least n leaves.
- Therefore, the height is at least $\lceil \log n \rceil$, meaning Bob must ask $\Omega(\log n)$ questions in the worst case.

Question 3 - Solution

Connection to the Optimal Strategy

- This lower bound is tight: Bob can achieve $\lceil \log n \rceil$ questions using a binary search strategy (e.g., always ask “Is it greater than the midpoint?”).

Question 4

We have an array A of n unique integers where there exists an index i (where $1 \leq i \leq n$) such that:

- $A[1] < A[2] < \dots < A[i]$ (strictly ascending),
- $A[i] > A[i + 1] > \dots > A[n]$ (strictly descending).

For example, in $A = [-5, 7, 10, 19, 32, 31, 30, 1, -1]$, $i = 5$.

We need to find i (the “peak”) using an **optimal** algorithm.



Question 4 - Solution (upper bound)

We use a **binary search**:

Given an index i , we can decide whether the peak is before/at/after position i by checking whether $A[i - 1], A[i], A[i + 1]$ is monotonically increasing / neither / decreasing.

Therefore, we can use binary search to find the peak in $O(\log n)$ time

Question 4 - Solution (lower bound)

To prove the algorithm is optimal, we show that peak-finding has a lower bound of $\Omega(\log n)$ by **reducing** searching in a sorted array (which has a known lower bound of $\Omega(\log n)$) to peak-finding.

Reduction:

- **Search in a sorted array Problem:** Given a sorted array A (in ascending order) and a target x , find the index k where $A[k] = x$.
- We construct a new array B whose peak is at index k
- Construct a new array B where:

$$B[i] = -|x - A[i]|$$

Question 4 - Solution (lower bound)

Why B has a peak:

- $|x - A[i]|$ is the distance between x and $A[i]$.
- Since A is sorted, as i increases, $A[i]$ gets closer to x until $A[i] = x$, then moves farther away.
- So $|x - A[i]|$ decreases until $A[i] = x$, then increases.
- Thus, $B[i] = -|x - A[i]|$ increases (becomes less negative) until $A[i] = x$, then decreases.
- The peak of B (where $B[i] = 0$) occurs at the index where $A[i] = x$.

Question 4 - Solution (lower bound)

- Searching in a sorted array requires $\Omega(\log n)$ comparisons (as shown in Question 3: a decision tree with n leaves has height $\log n$).
- If we could solve peak-finding in $o(\log n)$, we could solve search in a sorted array in $o(\log n)$ by:
 1. Constructing B ,
 2. Finding the peak in B ,
 3. Returning the corresponding index in A .

What is wrong with
this argument?

Question 4 - Solution (lower bound)

But wait! constructing B requires linear time.

What is the solution?

We will not explicitly create B or store it in memory. Instead, whenever the “peak” algorithm requires an element $B[i]$, we will compute in $O(1)$ time from $A[i]$.

Since the “peak” algorithm only examines $O(\log n)$ elements of B , the total running time is $O(\log n)$

Conclusion: It is not feasible to solve the peak problem more efficiently than the search problem. This establishes the desired lower bound.